



TTP: A Hardware-Efficient Design for Precise Prefetching in Ray Tracing

Yavuz Selim Tozlu[†]

*Electrical and Computer Engineering
North Carolina State University
Raleigh, USA
ystozlu@ncsu.edu*

Anshul Naithani[†]

*Electrical and Computer Engineering
North Carolina State University
Raleigh, USA
anaitha2@ncsu.edu*

Huiyang Zhou

*Electrical and Computer Engineering
North Carolina State University
Raleigh, USA
hzhou@ncsu.edu*

Abstract—Ray tracing (RT) is a 3D graphics technique that offers highly realistic visuals. It is becoming prominent and accessible as GPU vendors have integrated dedicated ray tracing acceleration hardware. However, tracing millions of rays through 3D scenes consisting of high numbers of triangles in real time is challenging and requires expensive hardware. The main bottleneck in RT workloads is the expensive Bounding Volume Hierarchy (BVH) traversal task, which is a large tree structure that encodes the 3D scene. BVH traversal is a memory-bound problem, as the GPU threads spend most of their time reading tree node data from memory.

In this work, we attack the memory latency bottleneck of ray tracing through prefetching. We propose a novel hardware prefetcher, named Tree Traversal Prefetcher (TTP), for ray tracing. The main idea is to leverage the existing tree traversal stack in the RT units for highly accurate prefetching. In particular, TTP prefetches nodes using the addresses already available on the hardware traversal stacks of each thread. For DFS (Depth-first search) based traversal, prefetches are generated when nodes are being popped consecutively from the traversal stack, potentially corresponding to upward traversal through the tree.

We evaluate TTP on a cycle-level simulator, Vulkan-sim 2.0, and show that it achieves 1.48x speedup on average (up to 1.89x) compared to the baseline, with nearly negligible hardware overhead. TTP achieves 98.92% average L1 accuracy, which is the ratio of the prefetched blocks being actually referenced by demand loads. The coverage, computed as the ratio of L1 miss reduction over baseline L1 misses, is 31.54%, correlating well with the achieved speedup.

Index Terms—GPU, Ray Tracing, Prefetching.

I. INTRODUCTION

Ray tracing is a modern 3D rendering technique that generates highly realistic graphics for both real-time applications like video games and offline rendering such as animations [7] [8] [20]. Unlike rasterization, ray tracing accurately models the lighting effects in a scene, providing life-like graphics [17]. The challenge, however, is that ray tracing incurs high computational costs. Recently, hardware vendors have introduced specialized units, RT units/cores, in GPUs to accelerate ray tracing [4] [5] [6] [16], prompting developers to adopt advanced ray tracing algorithms, thereby advancing real-time rendering [8] [10]. Beyond computer graphics, RT units have been exploited for more general purpose applications. [14] [22] [24] [26] [35] [44].

[†]Both authors contributed equally to this work.

Despite advances in ray tracing hardware, rendering complex 3D scenes in real time remains challenging. First, each frame requires tracing millions of rays to generate a high resolution picture. Second, although different rays traverse the 3D scene independently, thereby being massively parallel, each ray follows its own traversal path to bounce through objects to find intersections, leading to unpredictable and divergent behavior. Thirdly, during the traversal of the objects in a scene, each ray needs to access a large data set, i.e., the Bounding Volume Hierarchy (BVH). As a result, on-chip caches are usually not sufficient, and the memory wall is exposed as a key performance bottleneck.

BVH traversal makes up the bulk of ray tracing. A 3D scene is organized as a hierarchy of bounding boxes, which is stored as a tree in memory. The size of BVH trees can be in the order of gigabytes, depending on the scene complexity, thereby stressing the on-chip cache hierarchy. During traversal, an RT unit fetches tree nodes from memory, tests rays for intersections with boxes, and descends through the tree to find the closest hit primitive for each ray. As intersection tests and coordinate transformations are carried out in fast, fixed-function hardware, memory bottleneck is exposed as the primary challenge.

To better understand the performance of ray tracing workloads, Figure 1 shows the averaged distribution of thread status while executing a *trace_ray* instruction (the methodology is in Section V). The *trace_ray* instruction performs BVH traversal on the specialized GPU hardware, the RT unit. Within the RT unit, threads spend most of their time waiting for memory read requests to return, confirming that the memory wall is a significant bottleneck in ray tracing. To further analyze the memory performance of ray tracing workloads, we report the average DRAM bandwidth with and without our Tree Traversal Prefetcher (TTP) in Figure 2a. From the figure, we can see that most of the scenes do not fully utilize the DRAM bandwidth, suggesting ray tracing is mostly constrained by memory latency instead of bandwidth, also shown in previous work [40]. Additionally, underutilization of the bandwidth in the baseline suggests that there is sufficient headroom for a prefetcher.

In this paper, we aim to accelerate ray tracing by addressing its memory bottlenecks. We propose TTP to reduce the

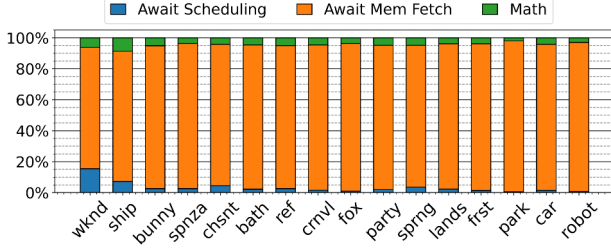
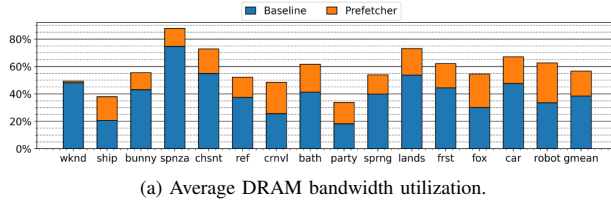
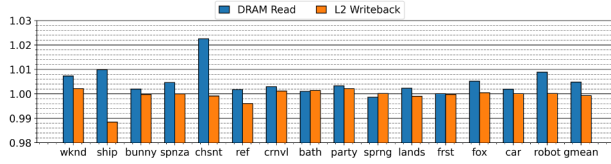


Fig. 1. Thread status distribution in an RT unit. Threads may be waiting for scheduling, waiting on a memory fetch, or performing math operations such as intersection tests or coordinate transformations. 128x128 resolution path tracing, 1 sample per pixel.



(a) Average DRAM bandwidth utilization.



(b) Total number of DRAM reads and L2 writebacks with TTP normalized to baseline.

Fig. 2. DRAM activity with and without (i.e., baseline) TTP.

memory access latency. The key idea of TTP is to leverage the existing per-thread traversal stack in the RT unit for generating prefetches based on the tree traversal trend, which is predicted from the stack push/pop operation sequences. Our analysis shows that TTP generates highly accurate prefetches (i.e., the prefetched data are demanded in a timely manner) with minor hardware changes to the RT unit. As seen in Figure 2a, TTP introduces a bandwidth overhead of 18.22% on average. On the other hand, the overall amount of data loaded from or written to DRAM remains nearly identical, as shown in Figure 2b. This shows that the increased bandwidth from TTP is due to the reduced execution time rather than extra data transfer from/to DRAM. Our results also show that TTP outperforms the state-of-the-art Treelet prefetcher [19], which was specifically designed for RT units.

In summary, this paper makes the following contributions:

- We propose TTP, a novel hardware prefetcher for ray tracing, which leverages the existing traversal hardware to prefetch data and hide the memory access latency.
- We study the traversal trends across the 3D scenes and show that TTP is highly effective in various ray tracing workloads and outperforms the state-of-the-art Treelet prefetcher.
- Our simulation results show that TTP achieves up to

1.89x performance improvement with a geometric mean of 1.48x.

- TTP supports both DFS- and BFS-based traversal, offering a new performance optimization opportunity by selecting different traversal algorithms for different workloads.

II. BACKGROUND

A. Ray Tracing

Ray tracing is a 3D rendering technique based on realistically modeling how light rays travel through space. High-performance graphics application programming interfaces (APIs) such as Vulkan [42] and DirectX [2] support ray tracing pipelines, which enable developers to build ray tracing applications. Ray tracing pipelines introduce new programmable shader types such as ray generation(*raygen*), closest-hit(*chit*), any-hit(*ahit*), intersection(*rint*) and miss(*miss*) shaders. Ray-gen shader is the beginning of the pipeline, where *primary* rays are generated by defining an origin and a direction in 3D space. Rays generated from the user's position in the world toward the scene are called *primary* rays. Rays that result from interactions with objects—such as reflections or refractions—are referred to as *secondary* rays. While the latest GPUs have built-in hardware support for testing intersections for triangles, procedural shapes such as spheres and cylinders require the *intersection* shaders to check if the ray intersects the object. If the intersected object is not opaque, then the *any-hit* shader is called for every intersection. Otherwise, at the end of the traversal, if the ray hits an object, the *closest-hit* shader is called. If the ray misses all objects, the *miss* shader is called.

Two types of ray tracing are supported, *closest-hit* and *any-hit*, for various purposes and applications. Closest-hit rays search for the primitive that is closest to the ray origin, whereas any-hit rays terminate their search upon finding any primitive that the ray intersects. Primary rays are always closest-hit rays; secondary rays are also closest-hit for path tracing, but any-hit for shadow and ambient occlusion shaders. Path tracing renders a 2D image of a 3D scene by tracing multiple rays per pixel and computing their resulting colors. Unlike hybrid approaches, which use ray tracing selectively for effects like shadows or ambient occlusion alongside rasterization, path tracing uses ray tracing exclusively to generate the entire frame. Shadow shaders determine the amount of direct light that reaches objects in a scene by tracing rays from the objects to the light sources. Ambient occlusion shaders estimate the amount of ambient light that reaches objects by tracing rays in random directions from the objects and calculating the degree of occlusion. Among the use cases of ray tracing, path tracing is the most intensive and time consuming workload. Typically, 4 to 16 bouncing rays are traced per pixel to determine its final color, compared to 2 or 4 rays traced for shadows and ambient occlusion. In addition, path tracing rays are always closest-hit, whereas other workloads use any-hit rays which are faster to trace.

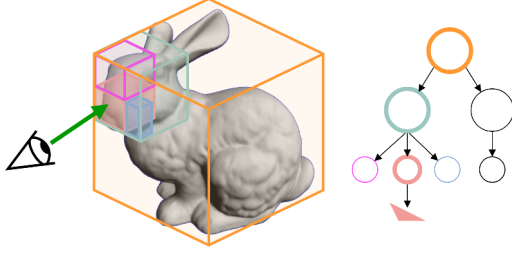


Fig. 3. Simplified BVH structure for Stanford Bunny. The green arrow represents the ray coming out of the camera, i.e. the eye. The ray intersects with the triangle inside the pink box.

B. BVH Traversal

To accelerate ray tracing, a tree-like structure called Bounding Volume Hierarchy (BVH) is employed, which consists of Axis-Aligned Bounding Boxes (AABB) that hierarchically capture scene primitives [25]. Rays first test outer boxes before deeper hierarchy levels, reducing the number of intersection tests. Primitives are leaf nodes of the tree, which are at the bottom. As rays test intersections and traverse deeper into the tree, they reach the leaf nodes and test against the primitives to determine if there is a hit. If a hit is found, remaining nodes are skipped if they are farther than the closest-hit primitive, which saves time.

As with any tree search problem, the tree can be traversed in depth-first or breadth-first fashion. For ray tracing, depth-first search (DFS) is often preferred as it quickly descends deeper and identifies a hit primitive, which allows the remaining nodes to be skipped if they are farther. A stack (last-in first-out or LIFO) data structure stores node addresses (not the node data itself) that will be read from memory next. Breadth-first search (BFS) is also a viable option. However, BFS processes nodes on the same level before going deeper, and takes longer to identify the first primitive hit. Therefore, it loses the opportunity to skip farther nodes. BFS uses a queue (first-in first-out or FIFO) structure to keep track of nodes, unlike the stack in DFS.

Figure 3 visualizes a simplified BVH structure and the corresponding tree for the Bunny object [9].

C. GPU Architecture and Hardware Accelerated Ray Tracing

GPUs are Single Instruction Multiple Thread (SIMT) computers. They support thousands of hardware threads that run in parallel, thus yielding massive computational throughput [28]. Figure 4 shows the baseline GPU model [39], with our proposed modifications highlighted in purple. A GPU consists of an array of Streaming Multiprocessors (SMs), as termed by Nvidia. SMs act as the main compute units where shader programs are executed. Each SM houses a register file, schedulers, hundreds of execution lanes, and an L1D cache. Shader programs are executed by groups of 32 threads, called "warps" by Nvidia, which operate in a lock-step manner. All the SMs are connected to an L2 cache via an interconnect, which is in turn connected to the DRAM memory controllers.

The latest GPUs also incorporate an RT unit in the SMs [5] [6]. RT units are introduced to accelerate the BVH tree traversal process, which accounts for the majority of ray tracing execution time [39]. Tree traversal involves fetching the BVH tree nodes from memory, testing for intersections, and fetching subsequent nodes based on the intersection results. The RT unit model used in this study is shown in Figure 4. A warp buffer stores the per thread traversal stack and ray metadata. At each cycle, a warp from the warp buffer is selected and a memory request from that warp is served. Duplicate requests within the same warp are coalesced into one request, and inserted into the memory access queue, which sends them to the memory hierarchy (i.e., L1 cache). In the meantime, memory responses are inserted into the response FIFO. Data in the response FIFO are fed into math units which perform ray-box, ray-triangle intersection tests or coordinate transformations depending on the node type. Finally, the warp buffers are updated and new node addresses are pushed to stacks based on intersection test results. Traversal continues until all threads in the warp have emptied their stacks (closest-hit), or found a hit (any-hit).

RT units accelerate BVH traversal by streamlining the memory reads and intersection tests. BVH traversal is carried out within the RT unit by a single CISC-like instruction, i.e., the *trace_ray* instruction. Dedicated memory schedulers and fixed-function intersection units inside the RT unit handle memory reads and primitive intersection tests internally, eliminating the need for explicit memory and compute instructions in the shader code.

During BVH traversal, GPU threads spend most of their time waiting for small pieces of tree data to arrive from memory [19]. As studied in the previous work [19], prefetchers can significantly mitigate this bottleneck by predicting which nodes are going to be read using heuristics and prefetching those nodes just before the threads need them.

D. Hardware Prefetchers

Hardware prefetchers, primarily studied for CPUs, include simple types like stream and stride prefetchers, which load data from consecutive addresses or detect strided memory accesses typically occurring in loops to reduce capacity and compulsory cache misses [23] [27]. Global History Buffer (GHB) prefetchers maintain a FIFO table of recent cache misses and use a linked list to traverse addresses for prefetching [37]. Indirect Memory Prefetchers (IMP) predict indirect memory accesses using address offsets and detect patterns with an Indirect Pattern Detector (IPD) [43]. While GHB and IMP handle irregular patterns, they struggle with the randomness of ray tracing workloads. Graph prefetchers aim to accelerate graph search algorithms but are less effective for the irregular nature of ray tracing [13]. Recent work [19] introduced the Treelet Prefetcher specifically designed for GPU ray tracing, which builds and prefetches treelets within the BVH tree. It requires a custom traversal algorithm and a rearranged BVH tree for optimal performance.

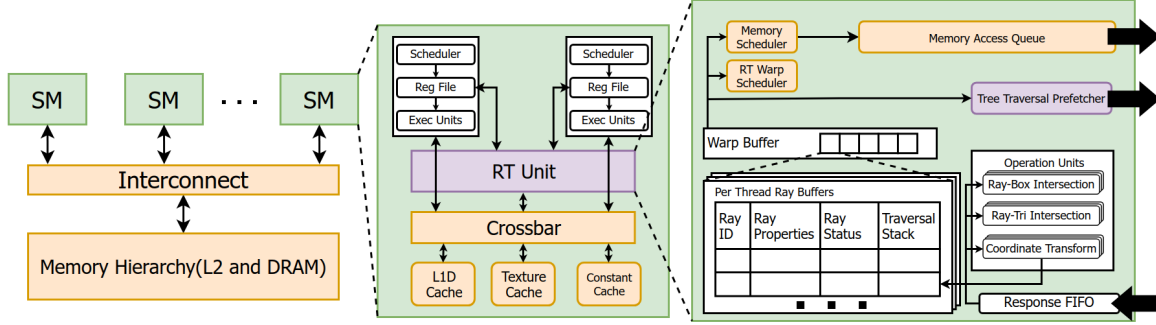


Fig. 4. Diagram of the GPU model used in this study. Purple blocks indicate the modified components. Redrawn from [39].

III. TRAVERSAL TREND IN RAY TRACING

We first study the BVH traversal trend in our ray tracing workloads to deeply understand the underlying traversal behavior and uncover prefetching opportunities. In particular, we look at the traversal stack activity to identify patterns which can be exploited.

As mentioned before, there are two types of hits that can be searched: closest-hit and any-hit. For closest-hit, the traversal is typically longer, as the whole tree must be covered to correctly identify the closest-hit primitive. This can lead to repetitive downward and upward traversal through the BVH tree, depending on ray origin and tree structure. Any-hit, on the other hand, requires identifying an intersected primitive regardless of its distance. We focus on *closest-hit* rays, as they are used in path tracing, which is the most intensive ray tracing workload. Algorithm 1 shows how DFS is carried out to find the closest-hit primitive. First, the root node is pushed to stack if its AABB is intersected (lines 1-2). Then, as long as the stack is not empty, node address at the top is popped and read from memory (lines 4-5). If it is an internal node, it contains the addresses and AABB coordinates of its child nodes, which are tested for intersection and pushed to stack if hit (lines 6-10). Otherwise, it is a leaf node which can be a triangle or a procedural geometry necessitating an intersection shader to be executed. If an intersected primitive is found, the closest-hit distance (min_thit) is updated (line 15). At the end of traversal, if a hit is found, registers are updated to indicate the hit, and geometry metadata is stored in memory to be read by the closest-hit or any-hit shaders.

A. Prefetch Opportunities in DFS

In DFS, nodes are pushed to and popped from top of stack. Consequent pushes indicate downward traversal towards the leaf nodes, while pops indicate upward traversal towards the root or across the same level. When consecutive pushes happen (downward traversal), it is difficult to predict which nodes will be needed, making prefetching highly challenging and speculative in nature, leading to wasted bandwidth. Since node pushing is determined by the results of intersection tests, it is often too late to prefetch the nodes by the time they are needed. In other words, top of stack may change depending

Algorithm 1: BVH Traversal using DFS to find the closest-hit primitive

Input: $ray, root_node_addr$
Output: $closest_node$

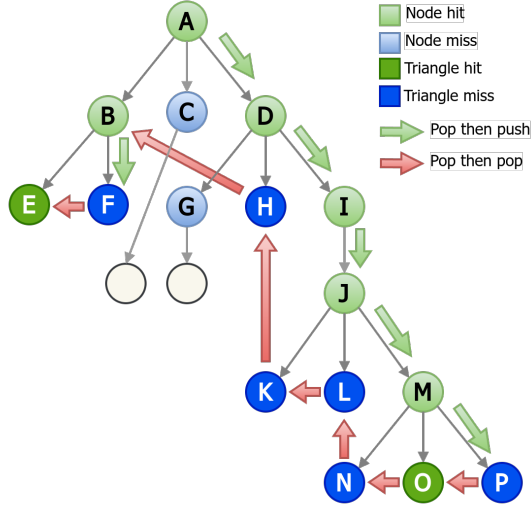
```

1 if  $ray$  intersects  $root\_node$  then
2    $stack.push(root\_node\_addr)$ ;
3 while  $stack$  is not empty do
4    $node\_addr \leftarrow stack.pop()$ ;
5    $node \leftarrow mem\_read(node\_addr)$ ;
6   if  $node$  is internal node then
7     for  $i = 0$  to 5 do // 6-ary tree
8        $thit \leftarrow$ 
9          $ray\_box\_test(ray, node.child[i].AABB)$ ;
10      if  $thit < min\_thit$  then
11         $stack.push(node.child[i].addr)$ ;
12  else // leaf node
13     $thit \leftarrow ray\_triangle\_test(ray, node)$ ;
14    if  $thit < min\_thit$  then
15       $closest\_node \leftarrow node$ ;
16       $min\_thit \leftarrow thit$ ;
17 return  $closest\_node$ ;

```

on the result of the current intersection test, so the next node to be read cannot be known until the test has completed. On the contrary, when consecutive pops happen (upward or same-level traversal), addresses of nodes to be read next are readily available in the traversal stack, and they can be trivially retrieved in a timely manner. This is the key insight driving our prefetcher design.

Figure 5a shows an example BVH tree traversal using the DFS algorithm. The traversal starts with the intersection test at the root node A . We assume that the ray eventually hits leaf nodes O and E . Therefore, when node A is popped and read from memory, the intersection test would show the ray hits the bounding boxes of B and D , whose addresses are pushed to stack. Then the address of D is popped and read from memory. Its children are tested for intersection and H , I are pushed to stack. This traversal continues until the whole tree is traversed, and nodes O and E are found as triangles that are hit. Note that the children of G and C are ignored because



(a) BVH Tree traversal with depth-first search. The ray eventually hits nodes *O* and *E*. Green arrows indicate a pop from the traversal stack followed by a push of its children. For example, node *A* is popped and read from memory. *A*'s node data contains AABB coordinates of its children *B*, *C*, and *D*, among which *B* and *D* test positive for intersection, and therefore pushed to stack. Red arrows indicate a pop from the stack followed by another pop. For example, *O* is popped after *P* is popped.



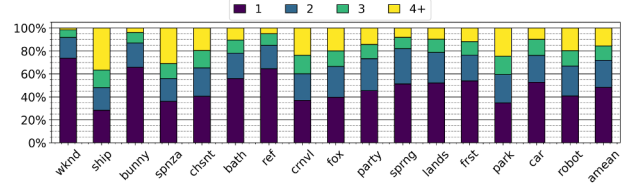
(b) Traversal stack right after *O* is popped. The stack contains the nodes' addresses. *N* is used instead of *&N* for simplicity.

Fig. 5. Example DFS BVH traversal and the corresponding traversal stack.

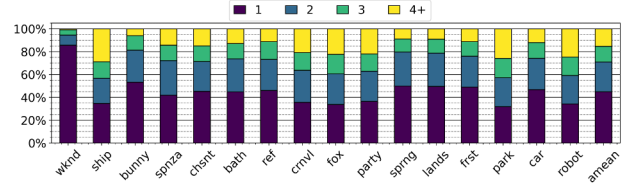
their bounding boxes are not hit and they are not pushed to the stack. Addresses of nodes *P*, *N*, *L*, *K*, *H*, and *F* are pushed to the stack because the ray intersects their bounding boxes, but misses the triangle inside them. Figure 5b shows the content of the thread's traversal stack right after the address of *O* is popped.

From this example, we make an important observation on the traversal trend in DFS, which is the frequent down-and-up traversal through the tree. In other words, a series of pop-push operations (traversing down the tree) are typically followed by a series of pop operations (traversing up the tree). In the example shown in Figure 5a, a tree branch, $A \rightarrow D \rightarrow I \rightarrow J \rightarrow M \rightarrow P \rightarrow O \rightarrow N$, is traversed until its tip is reached, then the upward traversal, $N \rightarrow L \rightarrow K \rightarrow H \rightarrow B$, begins. As the traversal stack already contains the addresses of these nodes, as shown in Figure 5b, we can prefetch them when we observe consecutive pops, meaning the thread is traversing the tree upward.

Note that although the node addresses (e.g., *N*, *L*, *K*, *H*, *B*) are in the stack, these nodes themselves have not been fetched and processed yet by the current thread (or ray). Therefore, prefetching them can reduce both cold misses if no other prior



(a) Pop streak frequencies.



(b) Composition of RT read misses in terms of pop streaks. The misses are those generating DRAM traffic, i.e., missing in both L1 and L2.

Fig. 6. Analysis of pop streaks.

rays (or threads) have fetched them, and capacity or conflict misses if they were fetched by other threads but evicted from the caches later on.

To see the impact of such traversal stack pop streaks, we extract traversal activity from the simulator. Note that during BVH traversal, every memory read request is a *pop* from a traversal stack. Therefore, we categorize the cache misses from RT reads by the position of the pop within a streak—i.e., whether it is the 1st, 2nd, 3rd, or 4th+ consecutive pop. The 1st pop (i.e., no consecutive pops) means downward traversal while consecutive pops(2, 3, 4+) correspond to upward or same-level traversal. We quantify both the frequency of these pop streaks and their cache miss rates in Figure 6. Although single pops, i.e. a pop followed immediately by a push, make up most of the stack activity, longer pop streaks make up a significant portion of all pops. Consequently, and more importantly, these longer pop streaks account for a large portion of all cache misses, making them prime candidates for prefetching.

B. Prefetch Opportunities in BFS

BFS is another option for tree traversal, where nodes at the same level are processed before descending deeper into the tree. In this case, the traversal stack would operate as a FIFO queue rather than a LIFO stack. Taking the BVH tree in Figure 5 as an example, after node *A* is popped and accessed, both nodes *B* and *D* will be pushed into a queue (i.e., queue content: *B*, *D*). After *B* is popped and accessed, its children *E* and *F* will be pushed into the queue after node *D* (i.e., queue content: *D*, *E*, *F*). Next, node *D* will be popped and accessed as it is the head of the queue.

While BFS is commonly used in traversing graphs [15] [34] [36], it is less effective for ray tracing compared to DFS, as it takes longer to identify the first hit and cannot skip as many nodes. Table I shows the average number of nodes per ray

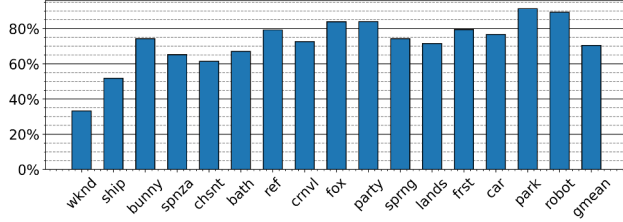


Fig. 7. Percentage of RT read misses where the node was in the traversal queue when a previously read node missed in caches.

visited by BFS and DFS for path tracing shaders, which trace closest-hit rays.

However, BFS has one important advantage over DFS: predictability. Because BFS uses a FIFO queue, the next node that will be read is known as long as the queue is not empty. New nodes are pushed to the tail of the queue, and the next node is popped from the head. This observation makes BFS an appealing candidate for prefetching as studied in previous work [13]. The key to accurate prefetching with BFS is timing. Although the address of the next node is readily available regardless of upward or downward traversal, prefetch must be done at the right time to avoid being too early or late.

With BFS, the traversal trend is always traversing the nodes on the same level before moving on to the next level. A prefetch opportunity exists as long as the queue is not empty. Following the example in Figure 5a, when the node B is to be popped, the queue content is the addresses of node B and D. As a result, when B is sent to the memory access queue, D can be prefetched. If both are cache misses, their latency can be overlapped.

In Figure 7, we show the percentage of RT read misses that satisfy the conditions, i.e., the queue is not empty and the demand access is a cache miss. In theory, these results show that, on average, 70.42% of all queue pops can be followed with an accurate prefetch, i.e. an upperbound on the prefetcher’s effectiveness. In practice, scheduling uncertainties and cache sizes will limit the effectiveness of the prefetcher.

IV. TREE TRAVERSAL PREFETCHER

A. Generating Prefetches with DFS

For DFS-based ray tracing, we propose to generate prefetches when the traversal trend is upward along the tree. To do so, we monitor the actions of the traversal stack: pushes and pops. With the first *pop* after a *push*, we generate 1 prefetch, which is the node at the top of the stack. With the second consecutive *pop*, we generate 2 prefetches using the top two nodes in the stack, and with the third *pop*, we are more confident about the trend and can be more aggressive in generating prefetches, e.g., by issuing up to 16 prefetches using the top 16 nodes in the stack. A *push* would reset the *pop* streak and stop generating prefetches. Figure 8 shows the state machine that implements this operation. This state machine can be implemented using 2 bits. There is no need to predict prefetch addresses as we simply use the content of

TABLE I
COMPARISON OF DFS AND BFS TRAVERSAL IN TERMS OF AVERAGE AND MAXIMUM NODES VISITED PER RAY. POSITIVE DIFF MEANS BFS VISITS MORE NODES.

Scenes	Average Nodes Per Ray			Max Nodes Per Ray		
	DFS	BFS	Diff	DFS	BFS	Diff
WKND	13.3	13.3	0.0%	73	73	0.0%
SHIP	43.3	46.6	7.6%	240	245	2.1%
BUNNY	11.3	12.5	10.6%	141	175	24.1%
SPNZA	38.0	51.5	35.5%	335	385	14.9%
CHSNT	59.0	137.4	132.9%	274	574	109.5%
BATH	18.4	21.1	14.7%	249	278	11.6%
REF	11.3	12.7	12.4%	235	291	23.8%
CRNVL	43.8	51.4	17.4%	487	492	1.0%
FOX	84.0	117.2	39.5%	568	874	53.7%
PARTY	34.8	37.4	7.5%	551	551	0.0%
SPRNG	31.4	38.7	23.2%	204	291	42.6%
LANDS	30.2	38.9	28.8%	527	535	1.5%
FRST	35.2	49.7	41.2%	355	943	165.6%
CAR	41.1	53.3	29.7%	303	582	92.4%
PARK	185.5	272.3	46.8%	1753	2071	18.1%
ROBOT	102.5	166.6	62.6%	1516	1841	21.4%
AVERG	49.0	70.0	42.9%	487.8	636.9	30.6%

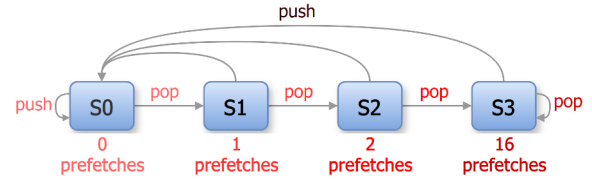


Fig. 8. State machine that generates prefetches.

the traversal stack. As each thread has its own traversal stack, our proposed TTP is a per-thread prefetching engine, sending prefetches when the warp is selected by the scheduler.

In the example shown in Figure 5a, *O* would be prefetched when *P* is popped from the stack (state S1), then *N* and *L* would be prefetched after *O* is popped (state S2).

The state machine can be adjusted for more flexibility, including the number of prefetches in each state and the number of states. However, as this state machine is one per traversal stack and, therefore, per thread, increasing the number of states may increase the hardware cost. Therefore, this paper uses the design in Figure 8 and studies the sensitivity of the parameters in our experiments.

We choose not to prefetch when a thread traverses down the tree. The reason is that we need to predict both intersection test results at different nodes and the descendent node addresses. Unless the prediction accuracy is high, such a scheme may incur high bandwidth/energy overhead and/or slowdown due to mispredicted paths. In contrast, prefetching for upward traversal does not require any address predictions.

B. Generating Prefetches with BFS

We propose a simpler prefetching scheme for BFS. With every pop from the traversal queue, up to *N* nodes are prefetched starting from the head. The parameter *N* determines the prefetch distance — that is, how far ahead in the queue the nodes are fetched. A larger *N* results in prefetching nodes

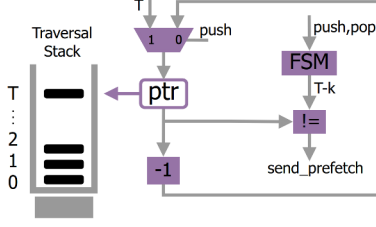


Fig. 9. TTP implementation. Top of the stack is denoted by T , and the bottom is 0. Purple blocks indicate newly added hardware structures. Only per-thread structures are shown.

that will be accessed further in the future. We explore with different values for N and report the results in Section VI-C.

C. Implementation

Figure 9 shows the implementation of TTP. Due to the simplicity of the underlying algorithm, very little hardware is required. We add an additional (2-bit) field to the warp buffer to implement the finite-state machine of each thread. Push and pop actions in the traversal stack orchestrates the finite-state machine, which in turn calculates the $T - k$ value, where T denotes the top of stack, and k is the prefetch distance, i.e. 1, 2 or 16. We add a pointer that points to the next address in the stack that will be prefetched. If a push happens, it is reset to T . If a pop happens, FSM is updated, which updates the $T - k$ value. Otherwise, with every prefetch, the pointer is decremented by one, which moves it to the next address in the stack. When the pointer reaches $T - k$, as determined by the comparator, prefetching stops. The pointer only resets to T upon a stack push, which prevents repeating prefetches from consecutive pops. Alternatively, each stack entry can have a flag bit to record whether the node address has been prefetched to avoid same entries being prefetched multiple times.

By default, prefetch requests are sent when there are no demand reads, i.e. demand reads take priority. In Section VI-B, we report results with different arbitration schemes where prefetches take priority if no prefetches were sent recently. Since the Vulkan-sim GPU model uses a sector cache model with sector size of 32B, prefetch requests for nodes that are larger than 32B are broken down into 32B chunks and sent one at a cycle, similar to demand reads.

When TTP is used with BFS, the prefetch distance k is a fixed value, N , rather than determined by the finite-state machine. Our experiments in Section VI-C analyze the choice of this parameter.

V. METHODOLOGY

We extend Vulkan-sim 2.0 [39] to model our proposed TTP, and host the open-source code at github.com/yavuz650/vulkan-sim. We use GPUWattch [30] for power analysis, which is included in the Vulkan-sim codebase.

The primary metric that we use for performance evaluation is the ratio of total number of simulation cycles, i.e., $\text{cycles_baseline}/\text{cycles_prefetcher}$. Baseline is the default RT unit that is shipped with Vulkan-sim.

Lumibench has 16 scenes with increasing geometric complexity, as summarized in Table II. 15 out of 16 scenes finish simulation without errors at 128x128 resolution. The **park** scene times out after 72 hours. Due to this, simulations for the **park** scene are ran at 64x64 resolution instead, which run into completion.

We compare our TTP with the state-of-the-art prefetcher for ray tracing, the Treelet prefetcher [19]. Both prefetchers are run at various resolutions using the same simulator configuration shown in Table III. For the Treelet prefetcher, we cloned and built the code available on Github [1] without any modifications. This repository has the necessary code that modifies the BVH tree to form treelets, perform treelet based traversal and prefetching. We enable the following options to turn on the Treelet prefetcher: `treelet_based_traversal`, `treelet_prefetch` and `keep_accepting_warps`. **chsnt** scene consistently crashes at every resolution when Treelet prefetching is enabled, and therefore not included in the results.

VI. RESULTS

A. Overall Performance Evaluation

We start with evaluating the overall performance of TTP. Figure 10 shows the normalized speedup, energy and power over the baseline. We see a geometric mean of 1.48x speedup (1.89x peak), 1.35x power and 0.91x energy consumption (i.e., 8.70% energy savings). Among the scenes, **wknd** is a very simple procedural scene featuring spheres and no triangles, offering limited room for improvement. We observe that TTP yields the highest speedups in scenes with long pop streak misses (**ship**, **crnvl**, **fox**, **party**, **robot**), as shown before in Figure 6.

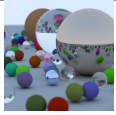
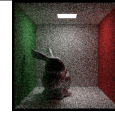
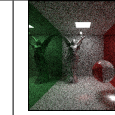
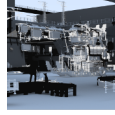
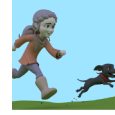
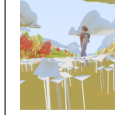
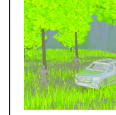
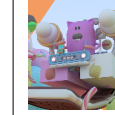
In addition, we perform a limit study by simulating with perfect upward and perfect downward traversals. For perfect upward traversal, 2nd and later pops after a push always hit in the L1 cache, and for perfect downward traversal, 1st pops after a push always hit in the L1 cache. Figure 11 shows the results, where geometric means are 1.79x and 1.35x. These results correlate well with the characterization in Figure 6.

TTP can also work with larger L1 data cache sizes. Figure 12 shows the normalized speedups (1.44x average for both) with cache sizes of 64KB and 128KB.

Figure 13 shows how the L1 and L2 RT read misses-per-kilo-instruction (MPKI) rates change with TTP. Both L1 and L2 misses reduce consistently (28.28% and 40.01%, respectively) across all scenes, verifying the effectiveness of TTP.

Figure 14 shows the accuracy and coverage of our TTP. *Accuracy* is the ratio of prefetched blocks that were accessed by a demand load. Average accuracies are 98.92% and 89.81% for L1 and L2 respectively. High levels of accuracy ensure that the cache is not polluted with unused data blocks. The accuracy is not 100% because in some rare cases, the prefetched data might end up being evicted before a demand load accesses it. *Coverage* is the ratio of useful prefetches to all RT misses of baseline. It indicates the proportion of misses that are prefetched and turned into hits. Averages are 31.54%

TABLE II
BENCHMARK SCENES FROM LUMIBENCH [32]. SCENE STATS TAKEN FROM [19].

Scene								
Label	wknd	ship	bunny	spnza	chsnt	bath	ref	crnvl
Tree Size(MB)	0.2	0.5	12.2	22	25.5	104.2	37.1	37.3
Depth	7	12	11	16	12	16	13	16
Scene								
Label	fox	party	sprng	lands	frst	park	car	robot
Tree Size(MB)	597.8	143.8	164.3	279.2	348.6	501.9	1,233.6	1,721.3
Depth	15	14	14	12	14	14	16	18

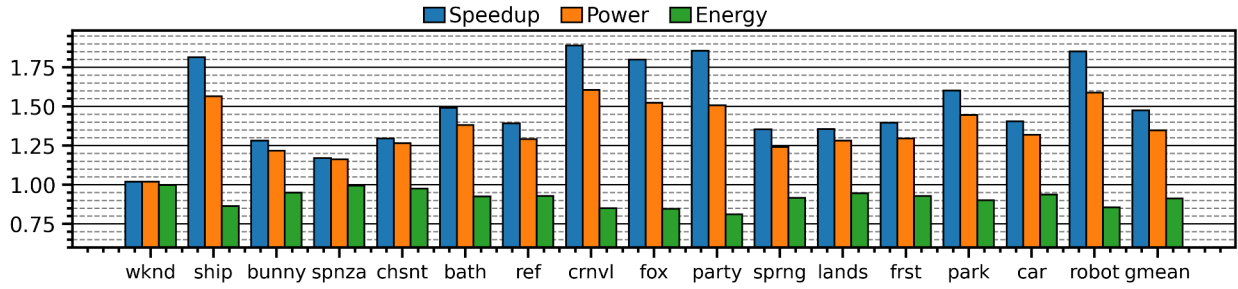


Fig. 10. TTP speedup (higher the better), power and energy (lower the better) with DFS traversal, normalized to baseline.

TABLE III
VULKAN-SIM HARDWARE CONFIGURATION

# Streaming Multiprocessors(SM)	8
Max. TBs per SM	16
Warp Size	32
Instruction Cache	128KB, 20 cycles
L1 Data Cache	32KB, Fully assoc. LRU, 20 cycles, 256 MSHRs
L2 Cache	512KB, 16-way assoc. LRU, 160 cycles, 768 MSHRs
Core, Interconnect, L2 Clock	1365 MHz
Memory Clock	3500 MHz
# of RT Units per SM	1
RT Unit Warp Buffer Size	4

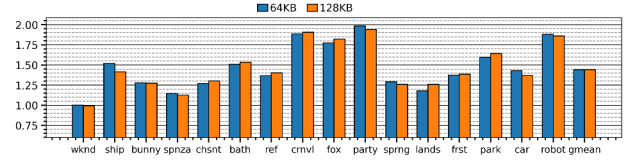


Fig. 12. Normalized speedsups with larger L1 data caches.

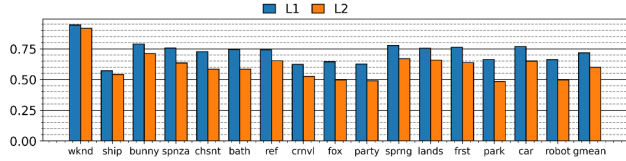


Fig. 13. RT read MPKI with TTP normalized to baseline. Lower is better.

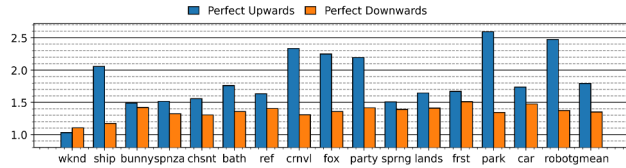


Fig. 11. Normalized speedsups with perfect upward and downward traversal.

and 33.46% for L1 and L2 coverages, respectively. Speedups are largely proportional to coverage. Higher coverage indicates that the BVH traversal for a particular scene exhibits the down-and-up traversal trend that our prefetcher performs well with.

To get a better understanding of the prefetcher efficiency, we look at the responses from caches. There are multiple possible outcomes of a cache access: (1) Hit, (2) hit in MSHR(Miss Status and Handling Register), or (3) miss in MSHR. If there

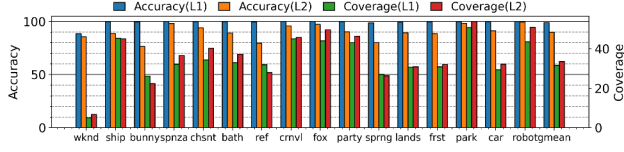


Fig. 14. Prefetcher accuracy and coverage with DFS traversal. Higher is better.

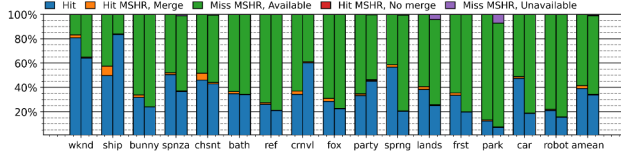


Fig. 15. L1 and L2 cache responses to prefetch requests. For each scene, first column is L1, second column is L2.

is an MSHR hit, the access request can be merged if a merge entry is available. If there is an MSHR miss, a new MSHR entry can be created if an empty entry is available. We present the cache response breakdown for prefetch accesses for both L1 and L2 caches in Figure 15. Note that the green bars, i.e. *Miss MSHR, Available*, are the ideal cache response type for a prefetch request. A prefetch request can only be useful if it misses in cache and MSHRs, and there is an empty MSHR entry available. We define the prefetcher efficiency as the ratio of prefetch requests that miss in caches and MSHR, i.e. the green bars in Figure 15. On average, the prefetcher efficiency is 58.56% and 64.85% for L1 and L2 respectively. Prefetches resulting in cache or MSHR hits are redundant but they do not pollute the cache.

By default, Lumibench uses 128x128 resolution for simulations. We increased the resolution to 256x256 to see the impact on TTP's performance. Figure 16 shows the normalized speedups at 256x256. Average speedup is 1.44x for 256x256, which is very close to the results at 128x128.

To further establish TTP's robustness, we simulate it with a different hardware configuration that has 30SMs, 64KB L1 cache and 3MB L2 cache. Figure 17 shows the normalized speedups in this configuration. Average speedup is 1.50x, and peak is 1.93x, which verify that TTP can work with different hardware setups.

B. Prefetcher Arbitration and Aggressiveness

We experiment with an arbitration scheme where prefetch requests are prioritized over demand requests if a fixed number

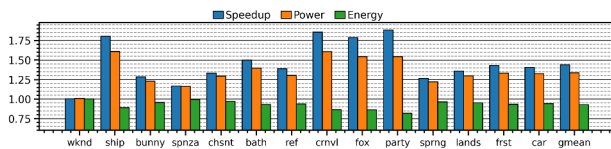


Fig. 16. Normalized speedup, power and energy at 256x256 resolution. **park** and **robot** scenes time out after 72 hours at this resolution.

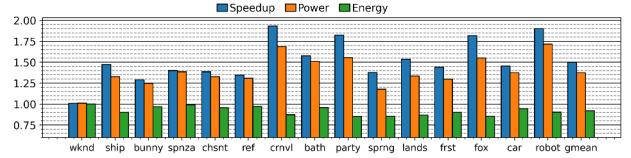


Fig. 17. Normalized speedup (higher is better), power and energy (lower is better) with a larger GPU model.

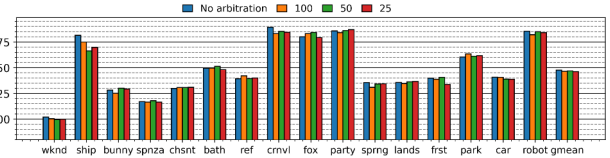


Fig. 18. Speedups for different prefetch arbitration configurations, normalized to baseline. Higher is better

of cycles have passed since the last prefetch request. Results are shown in Figure 18. We evaluate thresholds of 25, 50, and 100 cycles. On average, arbitration makes little to no difference.

Figure 19 shows the speedups for different prefetch intensities of TTP. The numbers correspond to the number of prefetches that are generated in each state, as shown in the state machine diagram in Figure 8. All configurations have similar overall performance although certain scenes prefer higher/lower intensity. This shows that TTP is not sensitive to this parameter.

C. TTP with BFS

Figure 20 shows the normalized speedup of TTP with BFS normalized to BFS without TTP. We experiment with 3 different prefetch distance values: 1, 2 and 4. TTP achieves an average of 1.85x, 2.05x, and 2.20x speedup for $N = 1, 2$, and 4 respectively. Longer prefetching distance is favorable, although the returns diminish quickly as N is increased. We use $N = 4$ for the rest of simulations for BFS.

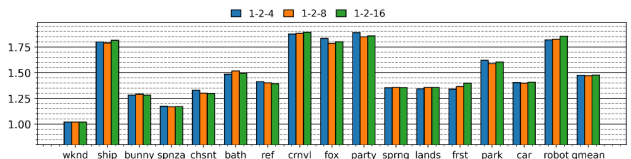


Fig. 19. Speedups for different prefetch intensities, normalized to baseline. Higher is better.

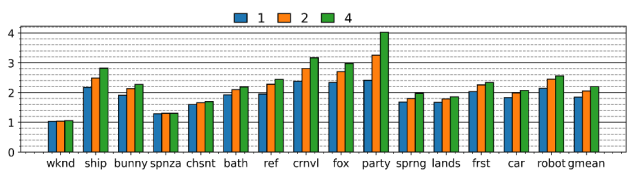


Fig. 20. Normalized speedup for different prefetch distance values with BFS.

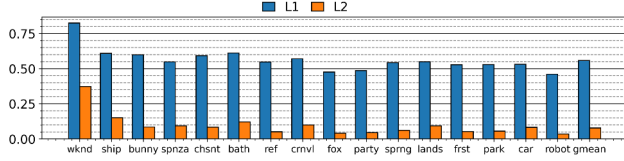


Fig. 21. RT read MPKI with TTP normalized to baseline, with BFS $N=4$. Lower is better.

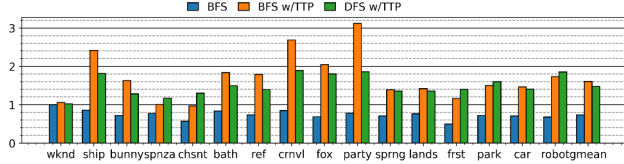


Fig. 22. BFS and DFS speedups normalized to DFS without TTP. $N = 4$ for BFS w/TTP. Higher is better.

Figure 21 shows normalized RT read MPKI. We note that both L1 and L2 misses are substantially reduced (44.10% and 92.04%, respectively), more so than the reductions in DFS. As expected, prefetching in BFS is much more effective since it is predictable.

We also compare DFS and BFS, with and without TTP. Figure 22 shows the speedups normalized to DFS. When no prefetching is involved, BFS is always slower than DFS. Interestingly, when TTP is enabled, BFS performs better than DFS on average, achieving 1.61x average speedup. Although BFS accesses more nodes than DFS, it has better cache performance and therefore processes the nodes quicker than DFS. The determining factor here is the *average-nodes-per-ray* metric, which we presented in Table I. DFS is faster in 5 scenes: **spnza**, **chsnt**, **frst**, **park** and **robot**. These 5 scenes also have the highest average-nodes-per-ray difference between DFS and BFS, and consequently, BFS loses its edge over DFS.

D. Comparison with Treelet Prefetcher

Treelet prefetcher is a GPU hardware prefetcher targeting ray tracing [19]. Treelet prefetcher requires the Treelet traversal algorithm, which involves pre-processing the BVH tree to divide it into treelets (sub-trees), and it directly replaces DFS. Once treelets are formed, the traversal is carried out in treelet granularity, i.e., all the nodes that are present within a treelet are traversed first before moving on to nodes in a different treelet. A treelet is prefetched when its root node is read. This requires adding additional information in the BVH tree nodes to identify which treelet a BVH node belongs to during traversal. As such, this requires modifications to the traversal algorithm and the BVH tree organization. Another disadvantage of the Treelet traversal is that it may access more nodes than DFS does, because instead of traversing deeper into the tree to find a triangle, it prioritizes traversing treelets. Figure 23 shows the speedups over baseline in path tracing for Treelet and TTP at 128x128 resolution. TTP has better performance in all the scenes. Treelet has 1.00x average

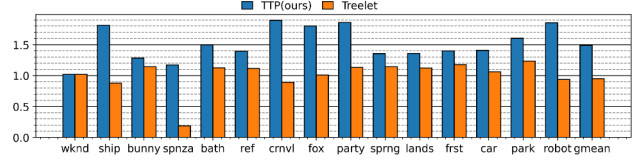


Fig. 23. TTP and Treelet prefetcher speedups normalized to baseline, 128x128 path tracing. Higher is better.

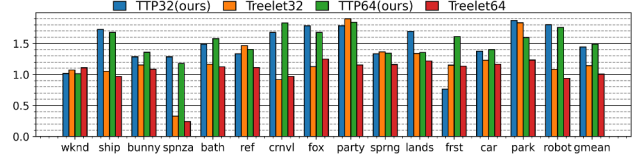


Fig. 24. Speedups (higher is better) of the two prefetchers across different resolutions. For each scene, first two columns are 32x32, the last two columns are 64x64.

speedup due to the performance loss in the scenes **ship**, **spnza**, **crnvl**, and **fox**.

To have a more comprehensive comparison, we simulate 32x32 and 64x64 frame resolutions. The results are presented in Figure 24. When the frame resolution is changed to 32x32, Treelet prefetcher shows 1.14x speedup, close to the results reported in the previous work [19], whereas TTP achieves 1.44x speedup. It should be noted that, at this low resolution, the SMs are under utilized due to the limited number of thread blocks. At 64x64, Treelet achieves 1.00x speedup, mainly due to performance regression in some scenes, whereas TTP maintains 1.49x speedup. These results show that TTP outperforms Treelet and is more robust across different resolutions. Treelet prefetcher generates excessive prefetches which consume too much bandwidth. This especially becomes a problem as we increase the resolution from 32x32 to 128x128, which results in more thread blocks and therefore higher bandwidth utilization. For example, in the **spnza** scene, average DRAM bandwidth usage goes up from 58% in baseline to 80% with Treelet at 128x128 resolution. It is likely that the excessive prefetches waste bandwidth and pollute the caches, leading to poor performance. Figure 25 shows the normalized number of total DRAM reads for Treelet prefetcher. On average, the DRAM traffic increases by 1.38x. For comparison, with our TTP, the total DRAM traffic does not change.

E. Comparison with Park et al. [38]

Park et al. [38] propose a prefetching scheme similar to TTP, the key difference being when and how often prefetches are

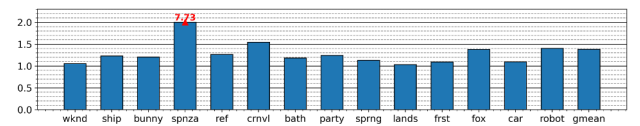


Fig. 25. Total DRAM reads for Treelet prefetcher normalized to baseline. Lower is better.

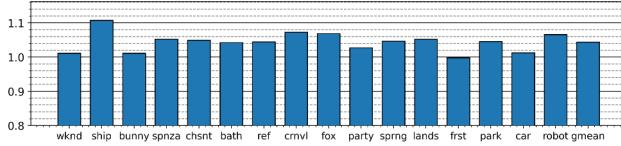


Fig. 26. Normalized speedups with Park et al.’s prefetching strategy. Prefetches are generated during a leaf node intersection test to overlap test latency with memory fetch latency.

triggered. Park et al.’s prefetcher aims to overlap fixed-function math latency (i.e. leaf node intersection tests) with memory latency. For this approach to be effective, math latency should be roughly on par with the latency of a cache miss. This depends on the math hardware, BVH tree format (i.e. how many primitives are packed in each leaf node), and the memory subsystem performance. The BVH tree used in this work is built by the open-source Embree library [3], which is what Vulkan-sim uses by default. The tree format has one primitive inside each leaf node, and the intersection latency in the simulator is configured as 8 cycles for a leaf node, which is remarkably faster than the latency of a cache miss (100-300 cycles). If the BVH tree format packed more primitives in each leaf node, then the intersection test latency would potentially increase. This could make it viable to overlap test latency with cache miss latency.

We model Park et al.’s prefetching strategy in Vulkan-sim to the best of our ability, and simulate using the same hardware configuration and resolution used for our TTP. We generate prefetches during a leaf node intersection test, and try to prefetch as many nodes as possible from the stack during that interval. Figure 26 shows the normalized speedups. We observed marginal speedups, averaging at 1.04x with a peak of 1.11x. The limited speedups are due to the timing of prefetches and how often the prefetcher is triggered. The intersection latency is too short to hide a long memory request latency; and prefetching only happens after a leaf node is accessed. In contrast, TTP covers all upward traversals.

F. Ambient Occlusion and Shadow Shaders

Lumibench features two additional ray tracing workloads: Ambient Occlusion (AO) and Shadow (SH) shaders. These shaders are used together with rasterization to improve visual quality, unlike path tracing which renders the frame by pure ray tracing. Both shaders first trace a primary ray from user’s position towards the scene and find the closest-hit point. Then, AO shaders estimate the ambient light that reaches crevices of objects by tracing 4 randomly directed rays from the point of intersection. SH shaders calculate shadows by tracing 2 rays from the hit point towards light sources. Secondary rays, also called shadow rays, are any-hit rays, i.e., they terminate traversal on first hit. In addition, they are more localized as they all share the same origin point (the intersection point of the primary ray). Therefore, they are already relatively fast to execute, leaving small room for improvement, unlike path tracing.

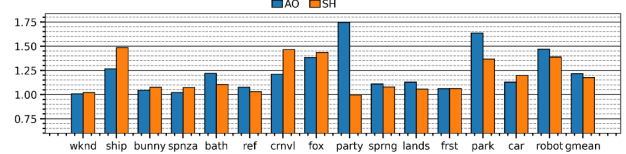


Fig. 27. TTP speedups (higher is better) with AO and SH shaders, 128x128 resolution. **park** is 64x64. Lumibench does not support AO and SH shaders in **chsnt** scene.

We simulated TTP with AO and SH shaders, and achieved 1.22x and 1.18x speedup on average respectively, as shown in Figure 27. **chsnt** is not included in the results, because Lumibench does not support AO and SH shaders in this scene.

G. Prefetcher Area Overhead

To estimate TTP’s area overhead, we implemented and synthesized the state-machine as described in Section IV using FreePDK45 [41]. We assume the same hardware configuration as in Table III, meaning there can be 4 warps in the RT unit, and each warp has 32 threads, therefore 128 state machines per SM. The total number of cells needed to synthesize 128 state machines is 1117. Each state machine has 2 bits of state, which requires 256 sequential cells. The remaining 861 cells are used for the combinational logic. On average, each state-machine requires $1117/128 = 8.7$ cells. To put this into perspective, as shown in Figure 4, just the *Ray Properties* field in the ray buffer would require $32 \times 3 \times 2 = 192$ bits of space per thread to store the ray origin and direction data. Therefore, we conclude that the area overhead of TTP is negligible in comparison to the existing hardware.

VII. RELATED WORK

A. Prefetchers

Lee et al. [29] propose a GPGPU prefetcher that extends the CPU stride/stream prefetchers for many-core architectures. Their design generates prefetches on a warp granularity by detecting strides on a per-warp basis. If multiple warps have the same stride, new warps can speculatively assume the same stride and generate prefetches. Liu et al. [33] use run-time information to dynamically adjust the prefetch amount per-thread and implement a filtering mechanism to discard harmful prefetches to avoid cache pollution. Ainsworth et al. [13] propose a graph prefetcher that targets BFS traversal. As graphs can be more complex than BVH trees, their prefetcher design entails complexities that TTP does not require. Specifically, their prefetcher snoops L1 accesses and determines which list (work, vertex, edge etc.) the address falls into. Then, it triggers a sequence of prefetches to fetch data from the lists, calculating a new address in between each prefetch. By contrast, TTP only monitors the hardware traversal stack activity, which directly stores the node addresses without multiple levels of indirection that general purpose graphs require. The Treelet prefetcher [19] is a GPU prefetcher that

targets ray tracing. They propose a treelet based tree traversal algorithm, and prefetch at treelet granularity. Our proposed TTP achieves higher and more robust performance gains than Treelet as discussed in Section VI.

B. Accelerating Ray Tracing

Hardware acceleration for ray tracing has been studied extensively [11] [12] [21]. Using Vulkan-sim as a baseline, specialized prefetchers and cache structures have been proposed to mitigate the memory latency sensitivity of tree traversal [19] [31]. Liu et al. [31] propose a dedicated cache structure that stores ray properties and intersected primitives for quick, speculative lookups to skip tree traversal. Chou et al. [18] propose a Treelet based RT unit design. Their solution is multifaceted, which involves virtual rays that increase the number of parallel rays in RT core, and dynamic shifting between traversal modes to accelerate traversal. Overall, Chou et al. [18] propose a more comprehensive and sophisticated solution, whereas TTP is a lightweight and low-overhead solution.

VIII. CONCLUSION

In this work, we propose a low-overhead, accurate hardware prefetcher, tree traversal prefetecher (TTP), for GPU accelerated ray tracing. TTP leverages the readily available traversal stack in hardware to accurately prefetch BVH tree nodes in a timely manner. For DFS based traversal, tree nodes are prefetched when consecutive stack pop operations happen, which indicate upward traversal. We show that such upward traversals make up a large portion of all RT read misses. As the per-thread traversal stacks store addresses of nodes that will be accessed next, this scheme requires no speculation of addresses, since the nodes in the stack will eventually be popped and read. TTP also readily supports BFS-based traversal, which is preferred for certain types of 3D scenes as well as general-purpose graph workloads.

We evaluate TTP on Vulkan-sim 2.0, a cycle-level GPU architectural model, and show that it achieves up to 1.89x speedup with a geometric mean of 1.48x in path tracing workloads, while saving the overall energy by 8.70%. To estimate the hardware overhead of TTP, we synthesize the RTL for state machines, which consume negligible space compared to existing hardware. We also compare our TTP with the state-of-the-art Treelet prefetcher specially designed for ray tracing. We simulate both prefetchers at various resolutions and show that TTP outperforms the Treelet prefetcher with much less hardware and software overhead.

ACKNOWLEDGEMENTS

We thank the anonymous reviewers for their valuable comments. The work is funded in part by NSF grants PHY-2325080 (with a subcontract to NC State University from Duke University), and OMA-2120757 (with a subcontract to NC State University from the University of Maryland).

APPENDIX

A. Abstract

This artifact provides the source code for the modified Vulkan-sim which has newly added configuration options to enable Tree Traversal Prefetcher(TTP). In addition, Python and shell scripts are provided to run all the necessary simulations and generate the plots and figures in this paper. To streamline the artifact evaluation process, we prepared a docker image with all the software dependencies installed. As simulations take a long time, we also included the raw simulation outputs that we generated and used for this paper.

B. Artifact check-list (meta-information)

- **Program:** Vulkan-sim, RayTracingInVulkan
- **Compilation:** gcc/g++, ninja, meson, cmake, nvcc
- **Run-time environment:** Ubuntu 20.04
- **Hardware:** 32+GB RAM, 35+GB Disk space
- **Metrics:** Number of cycles, average power consumption
- **Output:** Vulkan-sim simulation outputs, figures.
- **How much disk space required (approximately)?:** 30GB
- **How much time is needed to prepare workflow (approximately)?:** About 1 hour
- **How much time is needed to complete experiments (approximately)?:** 5 minutes to generate figures. One week for Vulkan-sim simulations, if ran in parallel
- **Publicly available?:** Yes
- **Code licenses (if publicly available)?:** Yes
- **Archived (provide DOI)?:** <https://doi.org/10.5281/zenodo.19394324>

C. Description

1) *How to access:* We provide the Docker image which has everything required to run the simulations and generate the figures. You can download it from Zenodo using the archived link.

2) *Hardware dependencies:* Only requirement is 32+GB of memory.

3) *Software dependencies:* Only a Docker installation is required. All software dependencies are installed in the Docker image.

D. Installation

Download the docker image from Zenodo, and start a container using the commands below,

```
$ docker load < ttp-isca2026-ae.tar.gz
$ docker run -it ttp-isca2026-ae:1.0 /bin/bash
```

E. Experiment workflow

Inside the container, we provide a shell script `ttp_simulations.sh` that has all the simulation commands needed to generate results. However, due to the large number of simulations, we do not recommend running the shell script directly, as it runs the simulations sequentially. Instead, depending on the resources, simulations should be run in parallel. The shell script simply serves as a reference for simulation commands. Workflow for launching parallel jobs depends on the system being used, therefore we cannot

provide a one-for-all script to launch parallel simulations. To launch a simulation in the container,

```
$ cd /home/root/vulkan-sim-root
$ source init_vars.sh
$ cd RayTracingInVulkan/build/linux/bin
$ ./RayTracer --scene 20 --width 32 \
--height 32 --samples 1 > ship_pt.log
```

We also provide all of the raw simulation logs and the Python scripts that we used to plot the figures in this paper. To generate the figures, following command can be used inside the container,

```
$ python3 figure1.py
```

This will generate `fig1.png` using the simulation logs under `ttp_raw_simulation_logs`. Other figures can be generated in a similar fashion.

F. Evaluation and expected results

Running the Python scripts will generate the figures using the existing simulation logs. To reproduce or replace the simulation logs, the simulation commands in the shell script can be used.

REFERENCES

- [1] “Code repo for Treelet Prefetching For Ray Tracing (MICRO 2023).” [Online]. Available: <https://github.com/ubc-aamodt-group/treelet-prefetching-for-rt>
- [2] “DirectX Raytracing (DXR) Functional Spec.” [Online]. Available: <https://microsoft.github.io/DirectX-Specs/d3d/Raytracing.html>
- [3] “Intel Embree.” [Online]. Available: <https://www.embree.org/>
- [4] “Intel® Arc™ Graphics Developer Guide for Real-Time Ray Tracing in...” [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/guide/real-time-ray-tracing-in-games.html>
- [5] “NVIDIA ADA GPU ARCHITECTURE.” [Online]. Available: <https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf>
- [6] “NVIDIA AMPERE GA102 GPU ARCHITECTURE.” [Online]. Available: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>
- [7] “Real-Time Ray Tracing.” [Online]. Available: <https://dev.epicgames.com/documentation/en-us/unreal-engine/hardware-ray-tracing-tips-and-tricks-in-unreal-engine>
- [8] “Real-time Raytracing for Interactive Global Illumination Workflows in Frostbite.” [Online]. Available: <https://www.gdcvault.com/play/1024801/>
- [9] “The Stanford 3D Scanning Repository.” [Online]. Available: <https://graphics.stanford.edu/data/3Dscanrep/>
- [10] “Unity real-time Ray Tracing.” [Online]. Available: <https://unity.com/ray-tracing>
- [11] T. Aila and T. Karras, “Architecture considerations for tracing incoherent rays,” in *Proceedings of the Conference on High Performance Graphics*, ser. HPG ’10. Goslar, DEU: Eurographics Association, Jun. 2010, pp. 113–122.
- [12] T. Aila and S. Laine, “Understanding the efficiency of ray traversal on GPUs,” in *Proceedings of the Conference on High Performance Graphics 2009*, ser. HPG ’09. New York, NY, USA: Association for Computing Machinery, Aug. 2009, pp. 145–149. [Online]. Available: <https://doi.org/10.1145/1572769.1572792>
- [13] S. Ainsworth and T. M. Jones, “Graph Prefetching Using Data Structure Knowledge,” in *Proceedings of the 2016 International Conference on Supercomputing*, ser. ICS ’16. New York, NY, USA: Association for Computing Machinery, Jun. 2016, pp. 1–11. [Online]. Available: <https://doi.org/10.1145/2925426.2926254>
- [14] A. Barnes, F. Shen, and T. G. Rogers, “Extending GPU Ray-Tracing Units for Hierarchical Search Acceleration,” in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’24. New York, NY, USA: Association for Computing Machinery, Nov. 2024.
- [15] A. Buluç and K. Madduri, “Parallel breadth-first search on distributed memory systems,” in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’11. New York, NY, USA: Association for Computing Machinery, Nov. 2011, pp. 1–12. [Online]. Available: <https://dl.acm.org/doi/10.1145/2063384.2063471>
- [16] J. Burgess, “RTX on—The NVIDIA Turing GPU,” *IEEE Micro*, vol. 40, no. 2, pp. 36–44, Mar. 2020, conference Name: IEEE Micro. [Online]. Available: <https://ieeexplore.ieee.org/document/8981896>
- [17] B. Caulfield, “What’s the Difference Between Ray Tracing and Rasterization?” Mar. 2018. [Online]. Available: <https://blogs.nvidia.com/blog/whats-difference-between-ray-tracing-rasterization/>
- [18] Y. H. Chou and T. M. Aamodt, “Treelet Accelerated Ray Tracing on GPUs,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS ’25. New York, NY, USA: Association for Computing Machinery, Mar. 2025, pp. 1334–1347. [Online]. Available: <https://dl.acm.org/doi/10.1145/3676641.3716279>
- [19] Y. H. Chou, T. Nowicki, and T. M. Aamodt, “Treelet Prefetching For Ray Tracing,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’23. New York, NY, USA: Association for Computing Machinery, Dec. 2023, pp. 742–755.
- [20] P. H. Christensen, J. Fong, D. M. Laur, and D. Batali, “Ray Tracing for the Movie ‘Cars,’” in *2006 IEEE Symposium on Interactive Ray Tracing*, Sep. 2006, pp. 1–6. [Online]. Available: <https://ieeexplore.ieee.org/document/4061539>
- [21] Y. Deng, Y. Ni, Z. Li, S. Mu, and W. Zhang, “Toward Real-Time Ray Tracing: A Survey on Hardware Acceleration and Microarchitecture Techniques,” *ACM Computing Surveys*, vol. 50, no. 4, pp. 58:1–58:41, Aug. 2017. [Online]. Available: <https://doi.org/10.1145/3104067>
- [22] Y. Feng, Y. Li, J. Lee, W. W. Ro, and H. Jeon, “HelioStat: Harnessing Ray Tracing Accelerators for Page Table Walks,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25. New York, NY, USA: Association for Computing Machinery, Jun. 2025, pp. 122–136. [Online]. Available: <https://dl.acm.org/doi/10.1145/3695053.3731011>
- [23] J. Fu, J. Patel, and B. Janssens, “Stride Directed Prefetching In Scalar Processors,” in *[1992] Proceedings the 25th Annual International Symposium on Microarchitecture MICRO 25*, Dec. 1992, pp. 102–110. [Online]. Available: <https://ieeexplore.ieee.org/document/697004>
- [24] L. Geng, R. Lee, and X. Zhang, “LibRTS: A Spatial Indexing Library by Ray Tracing,” in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP ’25. New York, NY, USA: Association for Computing Machinery, Feb. 2025, pp. 396–411. [Online]. Available: <https://dl.acm.org/doi/10.1145/3710848.3710850>
- [25] J. Gunther, S. Popov, H.-P. Seidel, and P. Slusallek, “Realtime Ray Tracing on GPU with BVH-based Packet Traversal,” in *2007 IEEE Symposium on Interactive Ray Tracing*, Sep. 2007, pp. 113–118. [Online]. Available: <https://ieeexplore.ieee.org/document/4342598>
- [26] D. Ha, L. Liu, Y. H. Chou, S. Go, W. W. Ro, H.-W. Tseng, and T. M. Aamodt, “Generalizing Ray Tracing Accelerators for Tree Traversals on GPUs,” in *Proceedings of the 57th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’24. New York, NY, USA: Association for Computing Machinery, Nov. 2024.
- [27] N. Jouppi, “Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers,” in *[1990] Proceedings. The 17th Annual International Symposium on Computer Architecture*, May 1990, pp. 364–373. [Online]. Available: <https://ieeexplore.ieee.org/document/134547>
- [28] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, “Accel-Sim: An Extensible Simulation Framework for Validated GPU Modeling,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, May 2020, pp. 473–486. [Online]. Available: <https://ieeexplore.ieee.org/document/9138922>
- [29] J. Lee, N. B. Lakshminarayana, H. Kim, and R. Vuduc, “Many-Thread Aware Prefetching Mechanisms for GPGPU Applications,” in *2010 43rd Annual IEEE/ACM International Symposium on*

- Microarchitecture*, Dec. 2010, pp. 213–224, iSSN: 2379-3155. [Online]. Available: <https://ieeexplore.ieee.org/document/5695538>
- [30] J. Leng, T. Hetherington, A. ElTantawy, S. Gilani, N. S. Kim, T. M. Aamodt, and V. J. Reddi, “GPUWatch: enabling energy optimizations in GPGPUs,” *ACM SIGARCH Computer Architecture News*, vol. 41, no. 3, pp. 487–498, Jun. 2013. [Online]. Available: <https://doi.org/10.1145/2508148.2485964>
 - [31] L. Liu, W. Chang, F. Demoullin, Y. H. Chou, M. Saed, D. Pankratz, T. Nowicki, and T. M. Aamodt, “Intersection Prediction for Accelerated GPU Ray Tracing,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’21. New York, NY, USA: Association for Computing Machinery, Oct. 2021, pp. 709–723. [Online]. Available: <https://doi.org/10.1145/3466752.3480097>
 - [32] L. Liu, M. Saed, Y. H. Chou, D. Grigoryan, T. Nowicki, and T. M. Aamodt, “LumiBench: A Benchmark Suite for Hardware Ray Tracing,” in *2023 IEEE International Symposium on Workload Characterization (IISWC)*, Oct. 2023, pp. 1–14, iSSN: 2835-2238. [Online]. Available: <https://ieeexplore.ieee.org/document/10289559>
 - [33] P. Liu, J. Yu, and M. C. Huang, “Thread-Aware Adaptive Prefetcher on Multicore Systems: Improving the Performance for Multithreaded Workloads,” *ACM Trans. Archit. Code Optim.*, vol. 13, no. 1, pp. 13:1–13:25, Mar. 2016. [Online]. Available: <https://doi.org/10.1145/2890505>
 - [34] L. Luo, M. Wong, and W.-m. Hwu, “An effective GPU implementation of breadth-first search,” in *Proceedings of the 47th Design Automation Conference*, ser. DAC ’10. New York, NY, USA: Association for Computing Machinery, Jun. 2010, pp. 52–55. [Online]. Available: <https://dl.acm.org/doi/10.1145/1837274.1837289>
 - [35] D. K. Mandarapu, V. Nagarajan, A. Pelenitsyn, and M. Kulkarni, “Arkade: k-Nearest Neighbor Search With Non-Euclidean Distances using GPU Ray Tracing,” in *Proceedings of the 38th ACM International Conference on Supercomputing*, ser. ICS ’24. New York, NY, USA: Association for Computing Machinery, Jun. 2024, pp. 14–25. [Online]. Available: <https://dl.acm.org/doi/10.1145/3650200.3656601>
 - [36] D. Merrill, M. Garland, and A. Grimshaw, “Scalable GPU graph traversal,” *SIGPLAN Not.*, vol. 47, no. 8, pp. 117–128, Feb. 2012. [Online]. Available: <https://doi.org/10.1145/2370036.2145832>
 - [37] K. Nesbit and J. Smith, “Data Cache Prefetching Using a Global History Buffer,” in *10th International Symposium on High Performance Computer Architecture (HPCA’04)*, Feb. 2004, pp. 96–96, iSSN: 1530-0897. [Online]. Available: <https://ieeexplore.ieee.org/document/1410068>
 - [38] J.-s. Park, W.-c. Park, J.-H. Nah, and T.-d. Han, “Node pre-fetching architecture for real-time ray tracing,” *IEICE Electronics Express*, vol. 10, no. 14, pp. 20 130 468–20 130 468, 2013.
 - [39] M. Saed, Y. H. Chou, L. Liu, T. Nowicki, and T. M. Aamodt, “Vulkan-Sim: A GPU Architecture Simulator for Ray Tracing,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct. 2022, pp. 263–281. [Online]. Available: <https://ieeexplore.ieee.org/document/9923844/citations?tabFilter=papers#citations>
 - [40] M. Saed, P. J. Nair, and T. M. Aamodt, “RayN: Ray Tracing Acceleration with Near-memory Computing,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’25. New York, NY, USA: Association for Computing Machinery, Oct. 2025, pp. 277–291. [Online]. Available: <https://dl.acm.org/doi/10.1145/3725843.3756067>
 - [41] J. E. Stine, I. Castellanos, M. Wood, J. Henson, F. Love, W. R. Davis, P. D. Franzon, M. Bucher, S. Basavarajiah, J. Oh, and R. Jenkal, “FreePDK: An Open-Source Variation-Aware Design Kit,” in *2007 IEEE International Conference on Microelectronic Systems Education (MSE’07)*, Jun. 2007, pp. 173–174. [Online]. Available: <https://ieeexplore.ieee.org/document/4231502>
 - [42] Vulkan, “Home | Vulkan | Cross platform 3D Graphics,” May 2024. [Online]. Available: <https://vulkan.org/>
 - [43] X. Yu, C. J. Hughes, N. Satish, and S. Devadas, “IMP: indirect memory prefetcher,” in *Proceedings of the 48th International Symposium on Microarchitecture*, ser. MICRO-48. New York, NY, USA: Association for Computing Machinery, Dec. 2015, pp. 178–190. [Online]. Available: <https://doi.org/10.1145/2830772.2830807>
 - [44] H. Zhang, Y. Zhang, and H.-W. Tseng, “RTSpMSPM: Harnessing Ray Tracing for Efficient Sparse Matrix Computations,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25. New York, NY, USA: Association for Computing Machinery, Jun. 2025, pp. 359–373. [Online]. Available: <https://dl.acm.org/doi/10.1145/3695053.3731072>